

Doctor-Notes Comprehension Service — Master Design (Draft)

Module: `hdh.modules.comprehension` · **Status:** open RFC — top-down design (decisions at this level; implementation deliberately deferred) · **Date:** 2026-08-11

Requesting comments — especially on §5 (keeping shared tables from breaking encapsulation), §6 (SapBERT's constrained slot), and the §14 drill-down queue. Substantive feedback earns a contributors-table entry.

Contributors

Name	Role	Contribution
Ajmal Mahmood	Author / Architect	Requirements, ontology strategy, top-down design

This is the master document. Each dependent ontology service (§10–§12) gets its own design document before it is built; this document fixes only the contracts between them.

Contents

- 1. [Purpose and contract](#)
 - 2. [Position in hdh](#)
 - 3. [The pipeline spine](#)
 - 4. [Dependent ontology services and the `OntologyService` protocol](#)
 - 5. [Shared tables without broken encapsulation](#)
 - 6. [Where SapBERT sits](#)
 - 7. [Specialty modularization](#)
 - 8. [Evaluation strategy](#)
 - 9. [Non-goals and safety](#)
 - 10. [Dependent service: SNOMED CT module \(placeholder\)](#)
 - 11. [Dependent service: RxNorm module \(placeholder\)](#)
 - 12. [Dependent service: LOINC module \(placeholder\)](#)
 - 13. [Phasing](#)
 - 14. [Open questions / drill-down queue](#)
-

1. Purpose and contract

Input: a free-text clinical encounter note (family medicine first; other specialties via packs, §7).

Output: a **coded structured record** — every clinical assertion in the note as a typed, ontology-linked entity. Each entity carries:

Field	Meaning
span	character offsets into the source note — provenance is non-negotiable ; an entity without a span does not exist
mention_type	problem · medication · lab/vital · procedure/order
code + ontology	the home-ontology concept (§4) with its ancestor context
assertion	negated · historical · family-history · uncertain · hypothetical · present
confidence	drives the review loop (§3 stage 7)

Export shape: FHIR `Composition` referencing `Condition` / `MedicationStatement` / `Observation` . Storage shape: schema-registry entities (`NoteRecord` , `NoteMention`) linked to `OntologyConcept` , so comprehension results are queryable by every other module.

The consumer contract: downstream consumers (agent, care-gaps, care-plan) read the comprehension, never the raw note. If something matters clinically and isn't in the structured output, that is a comprehension defect — not a reason for consumers to parse text themselves.

2. Position in hdh

A **specialized subagent** (the care-plan pattern): a LangGraph subgraph the main agent invokes as a tool (`comprehend_note`), also runnable standalone (`hdh comprehend --file note.txt / --mrn ... --visit ...`). Traced in the same trace DB; low-confidence mentions become checkpointed review points (interrupt/resume).

The **agent module is the prime consumer**: "what does this note say?", "code this note", "did the note mention anything we haven't captured as a diagnosis?". The **care-plan module is the endgame consumer**: note in → coded structured record → concerns/goals/interventions out — comprehension supplies the structured record the care-plan design (§ its 8.2) assumes.

3. The pipeline spine

Seven stages; the house rule throughout: **the LLM classifies, deterministic code decides**.

1 segment	note sections (SOAP; pack templates §7) – deterministic
2 extract	span-anchored mentions via closed-schema LLM structured output: find + classify, NEVER code
3 normalize	each mention → its home ontology's normalize() funnel (§4) → ranked candidate concepts
4 contextualize	negation / temporality / experiencer: NegEx-style rules first, LLM adjudicates only disagreements
5 disambiguate	ancestor-SET context picks among candidates – the H54 lesson as a stage (identical words, different subtrees, different patients)
6 assemble	typed record + FHIR export; entities written to

7 validate

registry tables with spans and confidence
every code must trace to BOTH a span and a catalog entry; Excludes1 conflicts across the note's code set
flagged; ungrounded output → retry with feedback;
low confidence → checkpointed human/agent review

Failure discipline mirrors the loaders: a stage that cannot meet its contract fails loudly; nothing half-comprehended is silently stored.

4. Dependent ontology services and the OntologyService protocol

Comprehension routes each mention type to its **home ontology**:

Mention type	Home ontology	Module	Status
Problems / diagnoses / findings	SNOMED CT (clinical primary) + ICD-10-CM (billing view via maps_to)	icd10cm  · snomed §10	icd10cm built · snomed designed (snomed-module.md)
Medications	RxNorm	rxnorm §11	needed
Labs / vitals / measurements	LOINC	loinc §12	needed
Procedures / orders	SNOMED procedures (CPT deferred — licensing)	snomed	designed — attribute edges in v1 carry intervention semantics

Settled decision — the OntologyService protocol. Every vocabulary module implements one typed interface (the GapFinder / LoadStage move applied once more), discovered like CLI_MODULES :

```
class OntologyService(Protocol):
    ontology: ClassVar[str] # "icd10cm" | "snomed_ct" | ...
    def lookup(code) -> Concept | None
    def ancestors(code) -> tuple[Concept, ...] # SET, not chain — a chain is
    def descendants(code) -> tuple[Concept, ...] # the degenerate tree case
    def synonyms(code) -> tuple[str, ...]
    def normalize(mention, context) -> tuple[Candidate, ...] # the funnel
    def subsumes(ancestor, descendant) -> bool # promoted to the protocol by
                                                # the SNOMED design: one closure
                                                # hit; disambiguation + care
                                                # gaps both want it cheap
```

Comprehension, the pattern compiler, and agent tools consume **the protocol** and never touch storage strategy. Consequences already settled:

- **Per-ontology hierarchy strategy is private.** ICD-10-CM keeps its materialized path (a tree, benched, earned). SNOMED stores N parent_of edges plus a **transitive closure table** — an implementation detail encapsulated inside the SNOMED module, exactly as every serious SNOMED deployment does. path is null for DAG ontologies.
- **Cross-ontology correlation** = curated maps_to edges (authority, confidence 1.0) where official maps exist; hybrid search (§6) at query time elsewhere; derived edges only with confidence < 1.0 and a derived authority tag. No pretend 1:1 crosswalks — the lossy ICD→SNOMED flattening this design exists to avoid.

5. Shared tables without broken encapsulation

`ontology_concepts` / `ontology_edges` are shared storage — which makes the tables a **second, unguarded interface**. Convention ("please call the service") does not prevent silent failure; this section is the plan that does.

The identified leak, concretely: two existing consumers query `path` directly. `patterns.py::_descend` compiles `parent_of depth "*" to path LIKE 'prefix%'`, and the agent's raw SQL tool advertises the `path` column. Both return **silently empty** results for DAG-ontology rows — wrong answers with no error, the worst failure class.

The plan, in order of strength:

1. **Route through the protocol.** `_descend` (and any future hierarchy consumer) dispatches to the owning module's `OntologyService`; direct `path` SQL outside a module's own service implementation is a design violation.
2. **Make the violation detectable.** The design-quality gate grows a check: outside `modules/<ontology>/`, no query may reference `ontology_concepts.c.path` / `hierarchy_depth` (waivable inline, as ever, with justification). Cheap AST scan; turns the convention into CI.
3. **Make misuse fail loudly, not empty.** Tree-only columns are null for DAG rows *by contract* — and the service raises `UnsupportedHierarchy` if a tree-strategy helper is invoked for a DAG ontology, rather than returning `[]`. Empty-but-wrong becomes impossible; loud-and-diagnosable replaces it.
4. **Tell the model the truth.** The agent's SQL-tool schema description is generated from metadata — it grows one line marking `path` as "ICD-10-CM only; use the ontology tools for hierarchy". The raw-SQL escape hatch stays (it's a feature), but the model is steered at the source.
5. **Verify at load time.** Each module's verify stage asserts its own invariants (tree: exactly one parent; DAG: acyclic + closure counts) so a mis-shaped load never reaches consumers.

Principle: **encapsulation enforced by interface + gate + loud failure — never by good intentions.**

6. Where SapBERT sits

One precise slot: **candidate generation inside** `normalize()` — nowhere else. Hybrid retrieval per mention: lexical FTS over the ontology's synonym index first; SapBERT bi-encoder nearest-neighbor (pgvector over embedded synonym strings) for the vocabulary gap ("sugar diabetes" → diabetes mellitus). SapBERT **proposes candidates; it never decides** — the deterministic scorer and the disambiguation stage (ancestor-set context, assertion attributes) select. H54 is the standing proof that embeddings alone must not choose: byte-identical descriptions, different patients.

Ships as the optional `[semantic]` extra: local CPU model, embeddings versioned in the load ledger, and **bench-gated** — lexical-only vs +SapBERT measured on linking accuracy (§8) before it is default-on. This operationalizes the ICD design's RFC Q10.

7. Specialty modularization

A `SpecialtyPack` is **data and configuration, not a code fork**: section templates, abbreviation/shorthand lexicon ("HTN", "PERRLA", "c/o"), mention-type priors, and a pack eval set. `family_medicine` ships first. `neurology`

follows as the modularity proof: exam-finding vocabulary (cranial nerves, reflex grading, NIHSS elements), neuro note shapes — plugged in exactly like `GapFinder`s, no pipeline changes.

8. Evaluation strategy

hdh's narrative module generates SOAP notes **from known ground truth** — the generator knows the true codes behind every synthetic note. That is an unlimited, perfectly-labeled corpus at zero annotation cost:

- **mention F1** (did extract find it), **linking accuracy** (right concept), **assertion accuracy** (negation/temporality) — per pack
- LLM-polished narrative variants (`--11m`) test robustness to phrasing drift the templates don't produce
- a small hand-curated golden set guards against the corpus's own bias (the generator's phrasing is the training distribution's cousin — open question §14)

9. Non-goals and safety

Educational demonstration over synthetic data. Not a clinical NLP product, not a medical device, never run on real PHI. Every output traces to a span and a catalog entry or it is rejected — the validator's grounding rule is the safety property. Not billing advice.

10. Dependent service: SNOMED CT module (placeholder)

Designed: see [snomed-module.md](#) — data sources and UMLS licensing logistics, first-time load, biannual updates, closure + attribute graph store, retrieval pipeline, and milestones. The paragraph below is the contract this master document fixed; the design honors it.

Own design doc before build. Fixed here: implements `OntologyService` ; loads RF2 (user-supplied under UMLS affiliate license — loader ships, data does not); N `parent_of` edges + private transitive closure; ~1.4M descriptions load as **first-class synonym rows** (the crown jewels for `normalize()`); `path` null. *Deferred to its doc:* attribute relationships (finding-site, causative-agent ...) as `edge_types` and when they earn load; release cadence; closure build/refresh strategy.

11. Dependent service: RxNorm module (placeholder)

Own design doc before build. Fixed here: implements `OntologyService` ; freely downloadable from NLM; the graph is ingredient → clinical drug → branded drug (dose form, strength) — a DAG, same closure treatment; brand/generic names as synonym rows; `normalize()` must handle "metformin 500 BID"-style dose-carrying mentions (span sub-structure — drill-down with §14's extraction schema).

12. Dependent service: LOINC module (placeholder)

Own design doc before build. Fixed here: implements `OntologyService` ; free with registration, not redistributable (loader ships, data does not); LOINC's structure is axes (component / property / system / method) more than

hierarchy — likely `properties.axes` + edges per axis, shallow closure; `normalize()` pairs a mention with its **value** and **unit** ("A1c 9.2%") for the Observation output.

13. Phasing

Phase	Delivers	Proves
1	<code>OntologyService</code> protocol + §5 items 1–4 retrofit on <code>icd10cm</code> (item 5 lands with each module's loader) — = <code>snomed-module.md</code> milestone A	encapsulation contract before a second ontology exists
2	SNOMED module — designed (snomed-module.md); build per its milestones B–D	DAG strategy, synonym index, protocol #2
3	RxNorm + LOINC modules	protocol scales; med/lab normalize
4	Comprehension pipeline on synthetic notes, lexical-only	the spine end-to-end, measured (§8)
5	Hybrid + SapBERT, bench-gated	Q10 answered with numbers
6	Subagent + <code>comprehend_note</code> tool + review loop	agent as prime consumer
7	Neurology pack	modularity claim proven

14. Open questions / drill-down queue

1. **The extraction schema** (next drill-down): what exactly is a mention — span granularity, composite mentions ("BP 142/88 on lisinopril" is a vital + value + medication + implied indication), list handling, section-scoped assertion defaults.
2. **Ground-truth bias**: synthetic notes as primary eval vs bootstrap-only — the generator's phrasing is kin to the templates being parsed.
3. ~~**SNOMED attribute relationships**: load with the module or defer?~~ **Resolved** by the SNOMED design (§1/§6 there): loaded in v1 as a generic `attribute` edge type — intervention semantics (method, procedure-site) demand them, per the thrombectomy analysis.
4. **Problem-list reconciliation**: when a note's comprehension disagrees with the chart's existing diagnoses, who wins and where is that recorded?
5. **Review-loop UX**: what does the checkpointed human review actually look like in a CLI-first tool?